

A PROJECT TITLE

ON

Travel Time Prediction Using Ridge And Lasso Regression

project report submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Name : B.Jamuna

Roll no. : 2520030408

Name : R.Pravalika

Roll no. : 2520030402

Under the Esteemed guidance of

Mr Shaik Asif

Assistant Professor, CSE

CSE



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in



DEPARTMENT OF CSE

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gendamasamma Road, Bowampet, Hyderabad, Telangana- 500 043
Phone No: 7815926816, www.klh.edu.in

Name of Title: TRAVEL TIME PREDICTION USING RIDGE AND LASSO REGRESSION

1. Abstract

Accurate travel time prediction plays a vital role in modern transportation and mobility systems. Applications such as ride-hailing platforms, GPS navigation tools, logistics planning, and urban traffic management all rely on the ability to estimate how long a trip will take under varying conditions. Poor travel time estimates can lead to inefficient routing, inaccurate fare calculations, and a degraded user experience. This project addresses this problem by developing a machine learning-based regression model to predict trip duration using historical trip data.

The dataset used for this project consists of trip-level records containing information such as pickup and drop-off coordinates, trip timestamps, and passenger counts. From this raw data, several meaningful features are engineered, including trip distance (calculated using geographic coordinates), hour of the day, day of the week, weekend indicators, and rush-hour flags. These engineered features often exhibit correlation with one another, which can cause instability and overfitting when using a standard linear regression model, particularly as the number of features increases.

To address this issue, the project applies two regularization techniques: Ridge regression and Lasso regression. Ridge regression applies an L2 penalty that shrinks coefficient magnitudes uniformly, improving model stability in the presence of multicollinearity without eliminating any features entirely. Lasso regression, on the other hand, applies an L1 penalty that can shrink certain coefficients exactly to zero, effectively performing automatic feature selection and producing a more interpretable model.

The dataset is split into training, validation, and test sets, and the regularization strength (α) for both Ridge and Lasso is tuned using k-fold cross-validation to identify the optimal value for each model. The performance of both models is evaluated using standard regression metrics, including Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 score, and compared against a baseline ordinary least squares regression model.

The results demonstrate that both Ridge and Lasso regression outperform the baseline linear regression model in terms of generalization to unseen data. Additionally, analysis of the coefficient shrinkage paths reveals which features contribute most significantly to travel time prediction, with Lasso providing clearer insight through its inherent feature selection capability. Overall, this project highlights the practical value of regularization techniques in building robust and interpretable predictive models for real-world transportation applications.

2. INTRODUCTION:

Transportation systems generate massive volumes of data every day — from ride-hailing trips and taxi journeys to delivery routes and public transit schedules. A central challenge across all of these systems is predicting **travel time**: how long a given trip will take from origin to destination. Accurate travel time estimation directly impacts user experience, operational efficiency, and business outcomes. Ride-hailing platforms such as Uber and Ola use travel time predictions to calculate fares and estimated arrival times; navigation apps like Google Maps use them to suggest optimal routes; and logistics companies rely on them to plan delivery schedules and driver allocation.

Despite its practical importance, travel time is influenced by a large number of interacting factors — trip distance, time of day, day of the week, traffic congestion, weather conditions, and route characteristics, among others. This makes it a non-trivial prediction problem, well suited to a machine learning approach where a model can learn patterns from historical trip data rather than relying on fixed rules or manual estimation.

3. SOFTWARE REQUIREMENTS:

3.1 Operating System

The project is platform-independent and can be developed and executed on any of the following:

Windows 10/11

macOS

Linux (Ubuntu/Debian-based distributions)

3.2 Programming Language

Python (version 3.9 or above) — chosen for its extensive machine learning ecosystem, readability, and wide community support for data science applications.

3.3 Development Environment / IDE

Google Colab — cloud-based, no local setup required, suitable for quick experimentation and sharing

Jupyter Notebook — for local interactive development and step-by-step analysis

Visual Studio Code — for writing and organizing modular, production-style Python scripts

3.4 Python Libraries and Frameworks

Data Handling and Preprocessing:

pandas — Loading, cleaning, and manipulating tabular trip data
numpy — Numerical operations and array handling

Machine Learning:

scikit-learn — Implementing Linear Regression, Ridge Regression, Lasso Regression, train-test split, cross-validation (GridSearchCV, RidgeCV, LassoCV), and evaluation metrics

Data Visualization:

matplotlib — Plotting graphs such as coefficient shrinkage paths and RMSE comparisons
seaborn — Statistical visualizations such as correlation heatmaps and distribution plots

Statistical Computation:

scipy — Supporting statistical computations during preprocessing and analysis

3.5 Optional Tools (For Extended Deployment)

joblib — Serializing and saving trained models

FastAPI — Building a REST API to serve model predictions

uvicorn — Running the FastAPI application server

MLflow — Tracking experiments, logging hyperparameters and evaluation metrics

3.6 Dataset Requirements

An equivalent dataset containing:

Pickup and drop-off coordinates

Trip timestamps

Passenger count

Trip duration (target variable)

3.7 Version Control

Git and GitHub — for source code management and project versioning



Signature of the Guide



Course Instructor